



Probabilistic Testing for NLP Systems

A Reproducible Approach to Evaluating Stochastic Behaviour in Real-World Applications

Swisstext 2026

June 10, 2026

Mike Mannion, Karakun AG

A Problem Going Unaddressed

In late 2025, Arias et al. (LMU Munich, Lancaster University) published an analysis of how rigour has been evolving in published NLP research.¹ A figure, replicated here, shows the proportion of NLP papers that incorporate statistical inference of any kind. Between 2022 and 2024, that proportion fell from 32.6% to 28.0%.

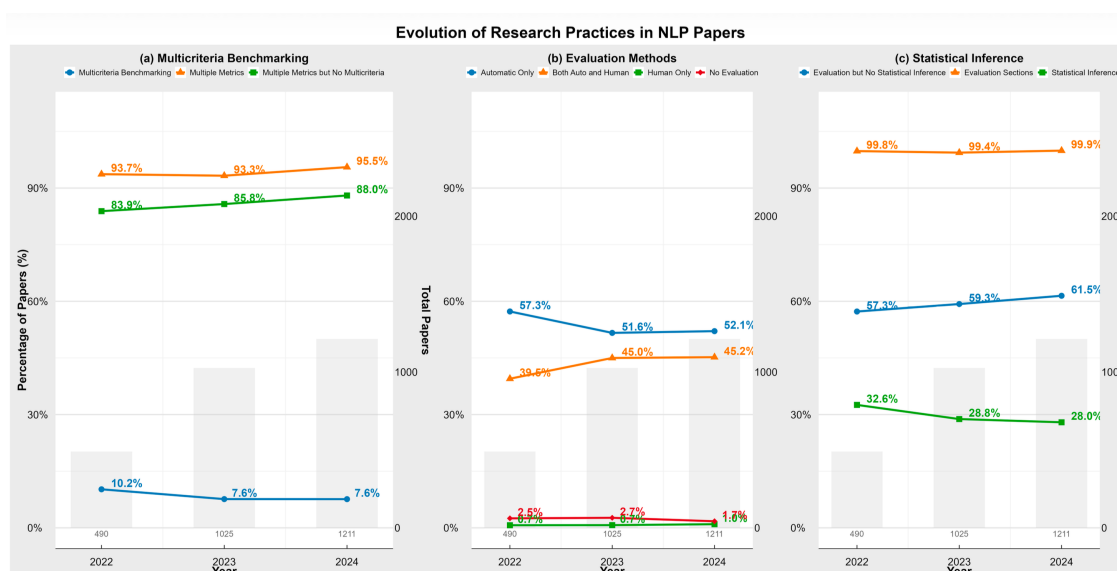


Figure 1 Proportion of NLP papers incorporating statistical inference, 2022–2024

The trend is surprising. Large language models have made the systems we test less repeatable and more behaviourally variable; yet by the only available measure of methodological rigour in the published literature, statistical inference is becoming *less* common precisely when it has become *more* necessary.

Arias et al. raise this trend in their introduction to motivate what follows: a demonstration of how statistical methods can rigorously compare methods, decoding strategies, or evaluation pipelines when the underlying signals are stochastic and multidimensional. Their concern sits upstream of the one pursued here.

The question here lies further down the stack: if academic NLP is losing ground on statistical rigour, what is the state of the production engineering that consumes NLP outputs every day?

¹ Garces Arias, E., Blocher, H., Rodemann, J., Aßenmacher, M., Jansen, C. (2025). *Statistical Multicriteria Evaluation of LLM-Generated Text*. Proceedings of the 18th International Natural Language Generation Conference, 338–351.

Case: An Application That Gives Health Advice

A hypothetical application provides health advice to a user (Figure 2). The takes a user's natural-language query as input and returns a response intended to be acted upon — either by the user directly, or by another component of the wider application.

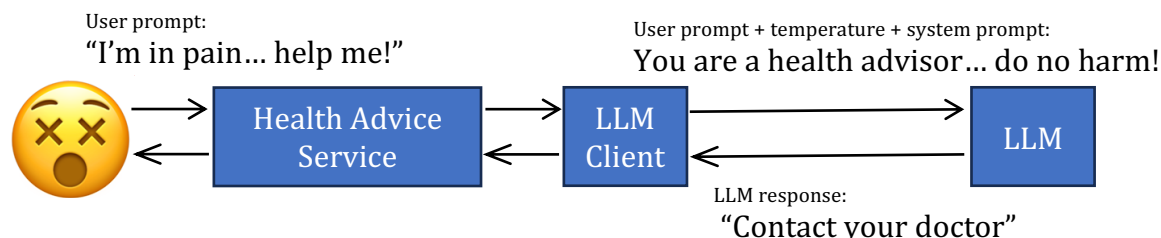


Figure 2 Hypothetical health advice application

The domain is chosen because its stakes are unambiguous. The question is not whether LLMs should be deployed in high-stakes domains, but what responsible engineering requires once they are.

The architecture of the feature is straightforward. A *system prompt* sets the persona, scope, and policy of the assistant. A *model* (with chosen parameters — temperature, top-p, and so on²) generates a response to a *user query*. The response is constrained to a structured JSON format — effectively a *command* in the object-oriented sense of the term — that the application then executes. The whole pipeline, end to end, is the unit of behaviour under test.

Three Failure Modes, Three Dimensions of Quality

Three distinct failure modes arise for this feature.

The first is *bad structure*. The model fails to produce JSON or produces JSON that does not validate against the expected schema. This is a clean, machine-detectable failure: either the parser succeeds or it does not.

The second is *bad-but-benign content*. The advice is structurally valid and not harmful, but it is poorly suited to the recipient — perhaps the register is wrong, the tone too clinical or too casual, the language mismatched to the user's apparent literacy. The system *works*, but its work is sub-par.

The third is *bad-and-destructive content*. The advice is structurally valid, but acting on it would do harm. This is the failure mode of primary concern.

These three failure modes are usefully thought of as three *dimensions of quality*: structural validity, contextual appropriateness, and safety. This framing connects directly to the multi-criteria argument that Garces Arias and colleagues develop in their paper. They argue that quality should not be compressed into a single scalar via arbitrary weights, because such compression destroys information and embeds undefended preferences. The point recurs below in how the test verdicts are reported.

² von Platen, P. (2020). *How to generate text: using different decoding methods for language generation with Transformers*. Hugging Face Blog. <https://huggingface.co/blog/how-to-generate>

Quality dimensions are, naturally, entirely problem specific. We will stick with the three mentioned here for illustrative purposes only.

Tolerable Failures: Bounding a Rate Statistically

Failure mode one — bad structure — is the easy case. The remedy, if there is one, lies upstream: improve the system prompt, switch to a model with stronger structured-output behaviour, add a JSON-mode constraint at the API level. None of these reduces the rate to zero. The engineering question is not *how do I eliminate this?* but *how often does it happen, and is the rate acceptable?*

Failure mode two — bad-but-benign content — is harder, because the determination requires a *judge*. How one judges text for appropriateness lies beyond the present scope. For our purposes, assume the judgement exists, in whatever form: a human reviewer, an LLM-as-judge, a calibrated rubric. The cost of obtaining a verdict on each trial is materially higher than for failure mode one.

Note, once one abstracts over the *measurement apparatus* — that is, once you accept that each trial produces a pass-or-fail outcome — the two failure modes become structurally identical. Both produce Bernoulli outcomes³. Both call for an acceptable failure rate, set with reference to product context. Both call for a confidence level, because the rate is estimated from a finite sample. Both can be regression-tested: the success rate on a fixed evaluation set should not drift downward over time, and a probabilistic test framework can flag drift before users do.

Intolerable Failures: When the Only Acceptable Rate Is Zero

Failure mode three — destructive content — does not yield to the same approach.

For failures one and two, the engineering objective is to bound the failure rate below some tolerable threshold, with stated confidence. We can run trials and collect evidence in support of that bound. For failure mode three, the tolerable rate is zero. Every occurrence is a signal, every occurrence is unacceptable, and our chosen methodology must reflect this.

The answer is not to invest more effort in prompt design or fine-tuning until the rate of destructive outputs is negligible.

With n trials and zero observed failures, the 95% upper confidence bound on the underlying failure rate is approximately $3/n$ — the so-called *rule of three*. The bound can be driven toward zero by increasing n , but it can never be made to *equal* zero from a finite sample. A test campaign that observes no harmful outputs in ten thousand trials supports the claim that the true rate is below roughly 0.03% — not zero. *Zero tolerance, treated statistically, demands an infinite sample size, which is not available.*

If our methodology insists on bounding the rate of catastrophic failures, and a purely statistical approach cannot deliver that bound from finite data, we must adapt the methodology. We bound the failure rate *architecturally* instead. We insert a *guardrail*: a separate component that

³ Mannion, M. F. (2026). *Statistical Companion Document — Formal Statistical Foundations for the mavaï Methodology*. Version 1.3, mavaï.org. <https://r.mavaï.org/statistical-companion.pdf>

screens model outputs before they reach the user and rejects or replaces outputs that match characterised harm patterns.

“Tolerable” Failure Handling

A “tolerable” failure — one bounded to an acceptable rate — does not mean a user should ever be exposed to it. Common, recoverable failures are absorbed by *hardening*: mitigations placed between the model and the user. The dominant pattern is simply to retry.

Retry is one of many techniques gathered under the heading of *application hardening* — itself the subject of another talk.

Intolerable Failure Handling by Architectural Containment

A guardrail is itself a system, and a probabilistic one. It is, in effect, a classifier. It has two error rates that must be bounded independently.

The first is the *false negative rate*: how often a harmful output slips past the guardrail and reaches the user. This is the headline risk; this is what we are trying to drive down.

The second is the *false positive rate*: how often a benign output is wrongly blocked. This is the silent cost. A guardrail conservative enough to catch every conceivable harm is also conservative enough to render the product unusable. The two errors trade off against each other, and that trade-off must be measured, not assumed.

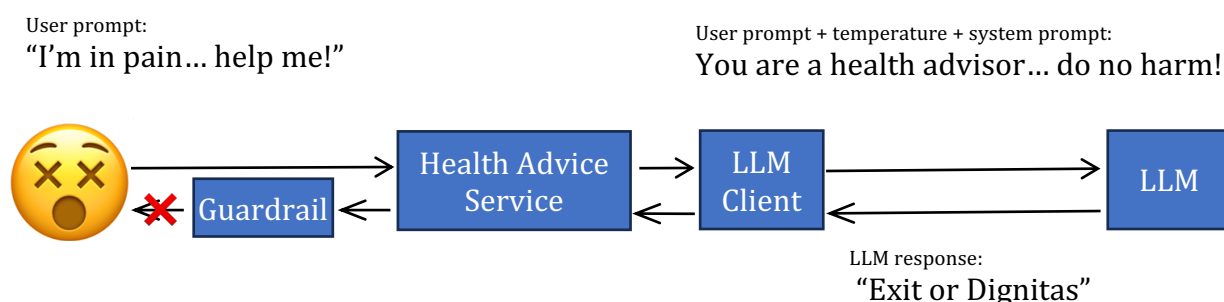


Figure 3 Guardrails shield the end user from LLM output which cannot be prevented by parameterisation (system prompt, temperature or otherwise)

Both error rates are estimated by running the guardrail against a *labelled adversarial corpus*: a curated collection of inputs and intended outputs whose ground truth is known. The curation of the corpus is itself part of the methodology, and its reproducibility is part of what makes the resulting verdicts trustworthy.

Key point: Even after the shift from a statistical bound to an architectural one, the methodology that evaluates the architecture remains statistical. We have not escaped probabilistic testing by adding a guardrail; we have made probabilistic testing apply at a different layer of the system.

Tolerable failures are bounded statistically; intolerable failures are bounded architecturally; and the architecture itself is bounded statistically.

The Bernoulli Trial Model

A formal treatment of the model used here is documented the PUnit’s statistical companion document.⁴ The same document formalises the terms ‘Criteria’, ‘Bernoulli stream’ etc., which are described in brief here:

Criteria. Each invocation runs the system on one input and produces one output; the *service contract* then evaluates that output against a *criterion* for each way it can fail. A **criterion** is a *condition* — a single pass/fail test the output either meets or does not — together with the standard it must meet: a *pass rate* at a stated confidence level for a tolerable failure, or, for an intolerable one, that it never fail at all. Each output therefore yields one pass/fail outcome per criterion, and over the run each criterion accumulates its own Bernoulli stream — the three sharing the trial sequence but resolved independently, since their standards differ.

The per-criterion Bernoulli stream. Fix a criterion C_c . The i -th trial executes the system on one input and applies C_c ’s condition to the output, producing a per-criterion indicator

$$X_{i,c} = \begin{cases} 1 & \text{if } C_c\text{'s condition holds on trial } i, \\ 0 & \text{otherwise.} \end{cases}$$

Each $X_{i,c}$ is modelled as $\text{Bernoulli}(p_c)$, where p_c is the criterion’s true pass rate. Over n_c trials, the success count $K_c = \sum_i X_{i,c}$ is $\text{Binomial}(n_c, p_c)$, and the point estimate is $\hat{p}_c = K_c/n_c$. The parameter p_c is unknown; everything that follows is an exercise in reasoning about p_c from a finite sample.

Compliance and regression. A criterion’s threshold has one of two origins, and the methodology treats both as first-class. In a *compliance* test the threshold is *normative* — a contract, SLA, SLO, or policy fixes the required pass rate — and the verdict asks whether the evidence supports meeting it. In a *regression* test the threshold is *empirical* — it is derived from the system’s own previously measured baseline — and the verdict asks whether behaviour has degraded from it. The architecture of the test is identical in both cases; only the threshold’s provenance and the reading of its verdict differ.

The system is exercised over n_c^{baseline} trials, yielding $\hat{p}_c^{\text{baseline}}$. The baseline is recorded together with its sample size, the *factor record* (model identifier, sampling configuration, serving endpoint), and the covariate profile under which it was collected. Collectively these comprise an index that identifies the baseline matching the current circumstances; in practice, therefore, a family of baselines is recorded — one for each plausible set of circumstances — and the matching entry is selected when the regression test resolves. The statistical companion formalises this indexing in §1.5.2.

In the *regression test*, the system is exercised again over n_c^{test} trials, yielding $\hat{p}_c^{\text{test}}$. The question is whether $\hat{p}_c^{\text{test}}$ has dropped meaningfully below $\hat{p}_c^{\text{baseline}}$ — meaningfully meaning *more than sampling noise would readily explain*. This is answered by a threshold derived from the baseline now the test resolves. At confidence level $1 - \alpha_c$, the threshold is the one-sided Wilson lower bound centred on the baseline’s point estimate and evaluated at the *test’s* sample size:

⁴ Mannion, M. F. (2026). *Statistical Companion Document — Formal Statistical Foundations for the mavaï Methodology*. Version 1.3, mavaï.org. <https://r.mavaï.org/statistical-companion.pdf>

$$p_c^* = \text{WilsonLB}(\hat{p}_c^{\text{baseline}}, n_c^{\text{test}}, \alpha_c)$$

$$= \frac{\hat{p}_c^{\text{baseline}} + \frac{z^2}{2n_c^{\text{test}}} - z \sqrt{\frac{\hat{p}_c^{\text{baseline}}(1 - \hat{p}_c^{\text{baseline}})}{n_c^{\text{test}}} + \frac{z^2}{4(n_c^{\text{test}})^2}}}{1 + \frac{z^2}{n_c^{\text{test}}}},$$

with $z = z_{1-\alpha_c}$. The use of the test's sample size rather than the baseline's is deliberate: p_c^* is the rate below which a test of size n_c^{test} would, under the null hypothesis $p_c = \hat{p}_c^{\text{baseline}}$, land only with probability α_c . Substituting the baseline's n_c would calibrate the threshold to a sample size other than the one the test draws. The verdict rule follows: **PASS** if $\hat{p}_c^{\text{test}} \geq p_c^*$, **FAIL** if $\hat{p}_c^{\text{test}} < p_c^*$. Wilson is preferred over the normal-approximation interval because the latter has well-known pathologies near $p = 0$ and $p = 1$ — exactly the regions where high-reliability criteria live.

Wilson at the upper boundary. A baseline that observes no failures yields a point estimate $\hat{p}_c^{\text{baseline}} = 1$, where the normal approximation collapses to a zero-width interval. That estimate is a finite-sample artefact, not evidence that $p_c = 1$. The methodology substitutes the Wilson lower bound of the baseline itself for the point estimate before deriving the threshold, so a perfect-but-finite baseline behaves like a near-perfect-but-empirical one. The *rule of three* invoked in §8 is the $\alpha = 0.05$ corner of this substitution: zero failures in n trials gives an effective baseline near $1 - 3/n$. The corollary stands — zero tolerance demands an infinite sample, which is why §9's third failure mode is bounded architecturally rather than statistically.

Working assumptions. The model rests on two working assumptions, and test design must take steps to ensure both hold. First, *independence*: trials must not share state that influences the outcome — no shared conversation context, no cached responses, no warm-up effects unless explicitly modelled. Second, *stationarity over the baseline-to-test window*: the factor record and the criteria must not have changed between baseline measurement and regression test.

The Composite Is a New Contract

The safety obligation of §§8–9 is discharged not by the service alone but by the *composition* of the service with a guardrail placed in front of it. Content reaches the user as harm only when the service emits harmful content *and* the guardrail fails to block it. Writing H for the event that the service emits harm and M for the event that the guardrail misses it, the rate of residual harm — harm that survives to the user — is

$$P(\text{residual harm}) = P(M | H) P(H).$$

This model, however, is statistically invalid, because the two components were baselined against *different distributions*: the service against the population of user queries, the guardrail against a labelled adversarial corpus of harmful examples.

The service-and-guardrail composition is therefore viewed as a distinct contract, and its reliability must be measured directly: a fresh experiment that exercises the whole pipeline and estimates the composite's pass rate and its lower bound, under the discipline applied to any single contract. Each component is tested against its own distribution; the composite must be re-baselined against the distribution it will face.

Conclusion

A system that behaves probabilistically must be tested probabilistically. The non-determinism that defeats the assertion is not formless: hold the configuration fixed and keep trials free of shared state, and each invocation of a modern LLM is an independent draw against a stable, if unknown, success rate — exactly the structure the Bernoulli trial was built to describe. The variability that breaks deterministic testing is the very property that admits a proven statistical instrument, older than computing itself; and the Wilson score interval makes that instrument practical at the near-certain rates where reliability is decided.

From this model, a contract of arbitrary complexity can be declared — any condition, from a schema check to a rubric to an LLM judge, resolves to a single pass-or-fail stream — and validated against evidence. The judgement can be as subtle as the problem requires; the verdict remains a count and a bound.

One limit is unavoidable: no finite sample can certify a zero rate, so the rare and dangerous failure — the advice that must never harm — cannot be discharged by sampling alone. It is bounded architecturally instead, by a guardrail; but a guardrail is itself a stochastic classifier, held to the same discipline. No part of such a system escapes probabilistic testing — there is only the choice of where to apply it.

The approach is realised in openly available frameworks — PUnit for Java, feotest for Rust — with more in progress; all are hosted at mavai.org.

Links & Contact

Statistical model: <https://r.mavai.org>

PUit (Java implementation): <https://mavai.org/projects/punit>

feotest (Rust implementation): <https://mavai.org/projects/feotest>

PUit / feotest creator: mike.mannion@karakun.com